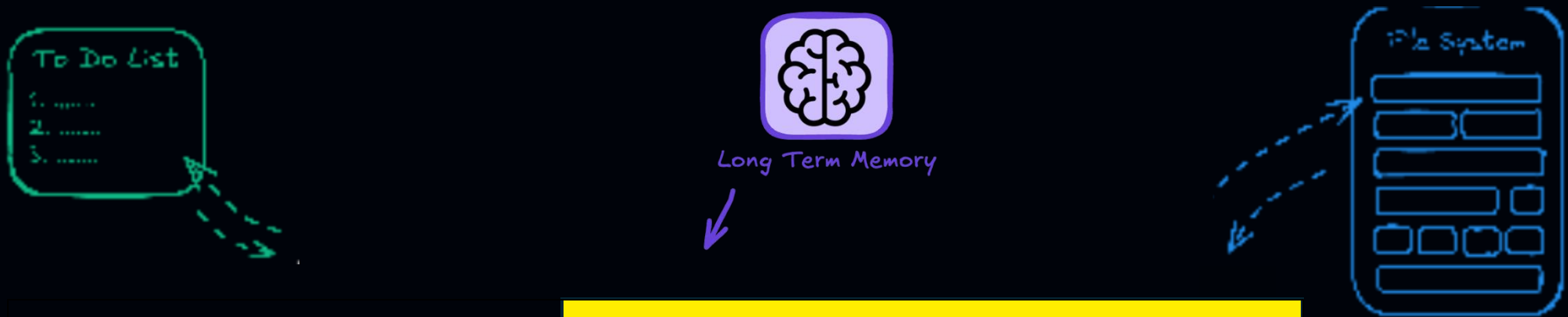
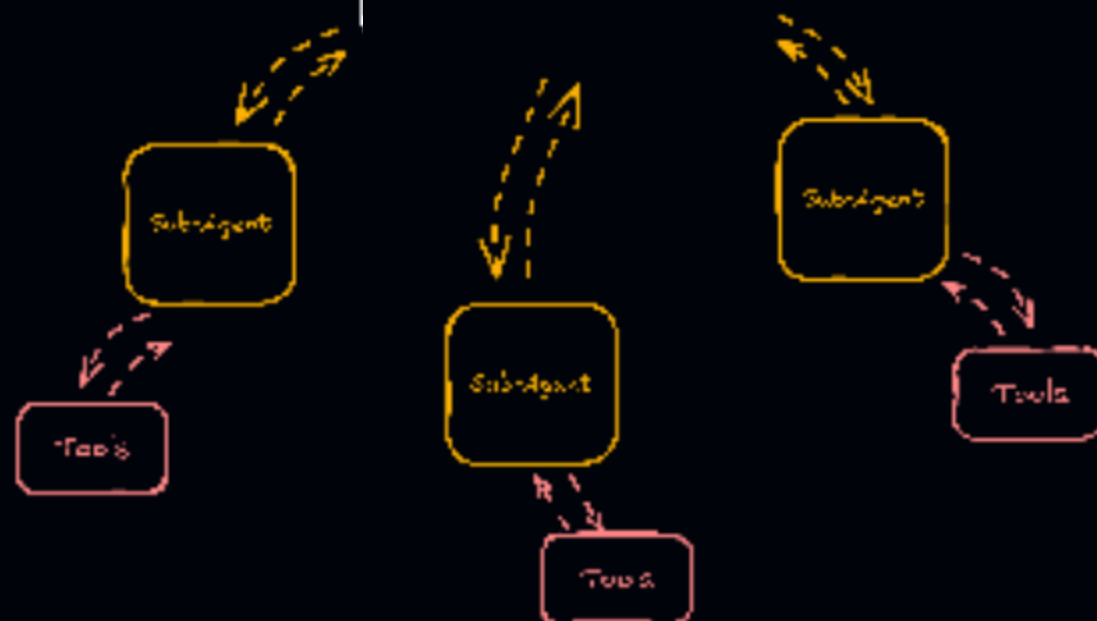




Deep Agents & Long-Term Memory



Ever Wondered How...



OpenAI Deep Research

Can work for over an hour, perform 100s of web searches and overflow GPTs context window many times over



Claude Code

Can navigate huge codebases without using semantic search



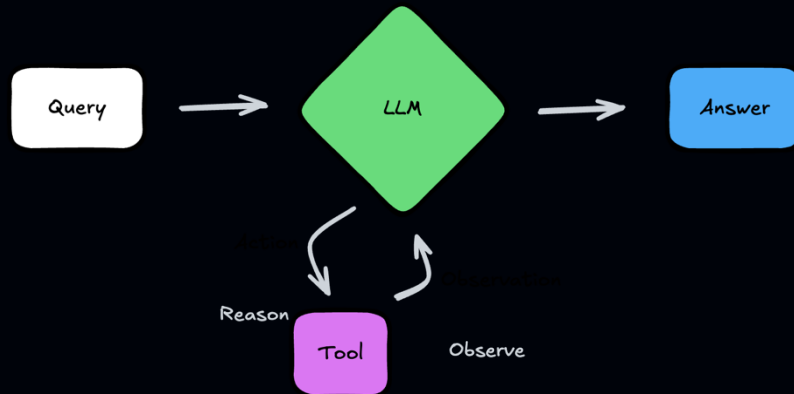
Manus Agent

Can use 50+ different tools including computer use

Whilst simple AI agents get stuck after 7-8 steps...

Deep Agents

1.0 Shallow Agent

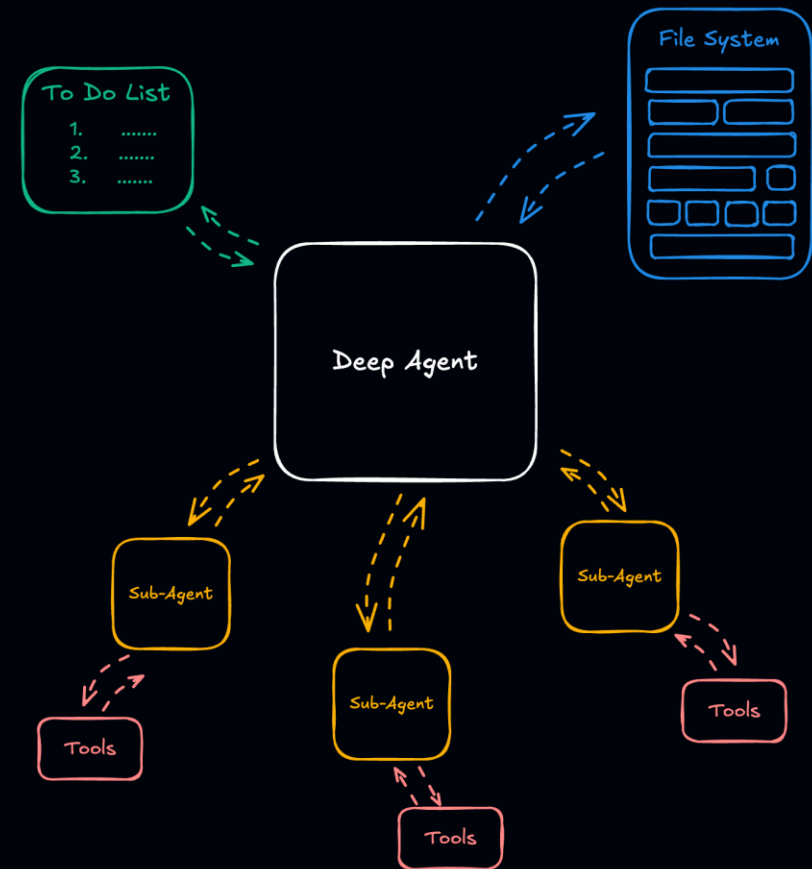


How Shallow Agents Fail:

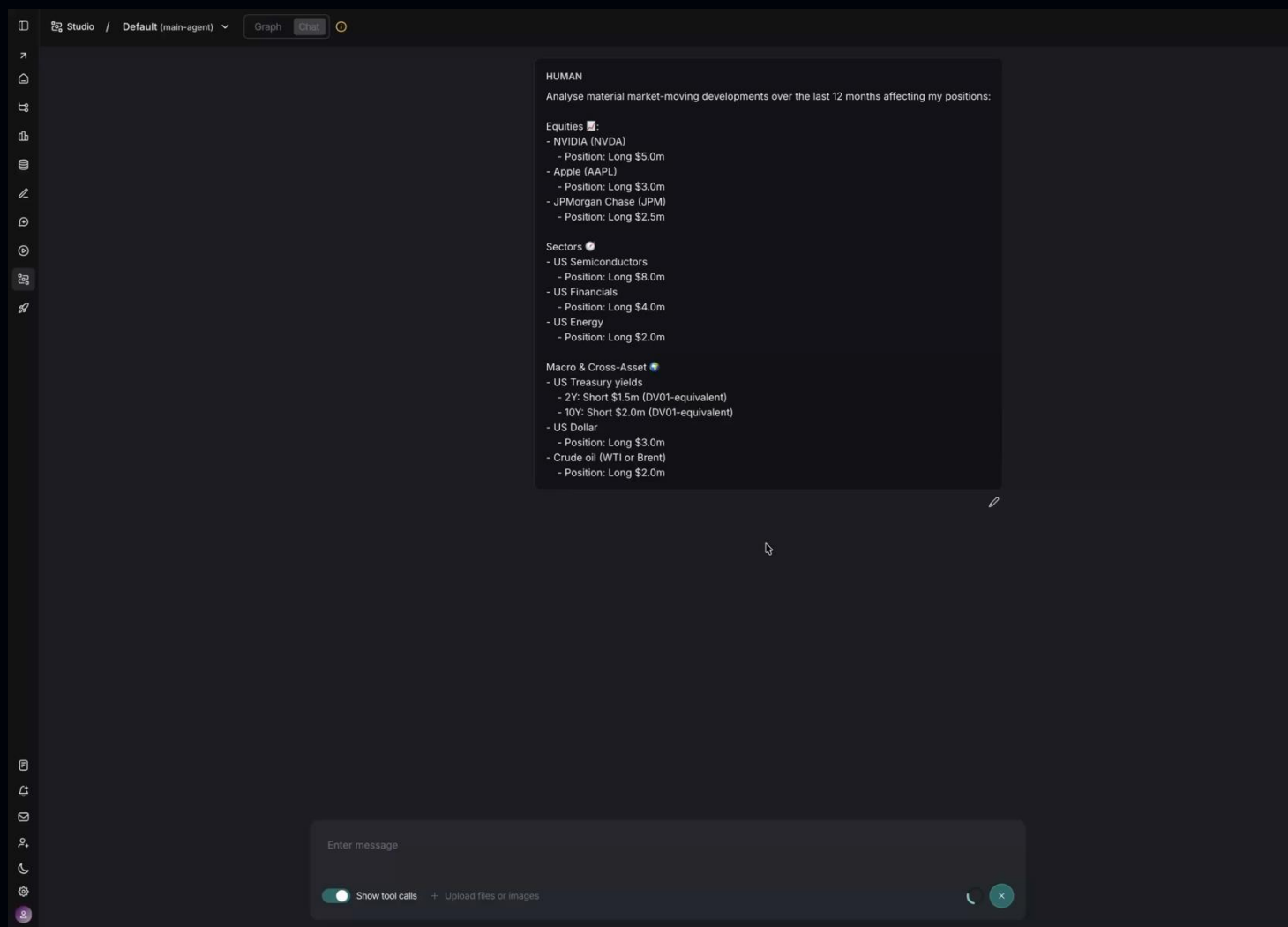
- Context Overflow: The history fills up with tool outputs (HTML, messy data), pushing instructions out of the context window.
- Loss of Goal: Amidst the noise of intermediate steps, the agent forgets the original objective.
- No Recovery mechanism: If it goes down a rabbit hole, it rarely has the foresight to stop, backtrack, and try a new approach.

We will build this
but better
with long-term memory

2.0 Deep Agent



Demo



Deep Agent in Real Time: maintaining to-do list, learning form historical memories and spawning dozens of sub-agents...

Agenda

1. Fundamentals of Context Engineering [Theory]

2. LangGraph Tutorial [Code]

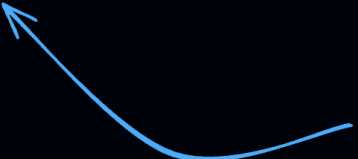
3. Deep Agent with Long-Term Memory [Theory + Code]

4. Use Cases

- a. Long-Term Memory
- b. Deep Agents

Whilst deep agents are very complex...

long-term memory is very simple
and can be applied to any AI system
with huge benefits...



Context Engineering

**Andrej Karpathy** ✓
@karpathy

+1 for "context engineering" over "prompt engineering".

People associate prompts with short task descriptions you'd give an LLM in your day-to-day use. When in every industrial-strength LLM app, context engineering is the delicate art and science of filling the context window with just the right information for the next step. Science because doing this right involves task descriptions and explanations, few shot examples, RAG, related (possibly multimodal) data, tools, state and history, compacting... Too little or of the wrong form and the LLM doesn't have the right context for optimal performance. Too much or too irrelevant and the LLM costs might go up and performance might come down. Doing this well is highly non-trivial. And art because of the guiding intuition around LLM psychology of people spirits.

PHILSCHMID

SEARCH · BLOG · PROJECTS · NEWSLETTER · ABOUT ME

The New Skill in AI is Not Prompting, It's Context Engineering

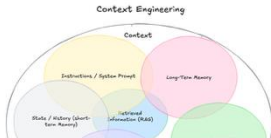
June 30, 2023 · 5 min read

Context Engineering is new term gaining traction in the AI world. The conversation is shifting from "prompt engineering" to a broader, more powerful concept: **Context Engineering**. **Tabularius** describes it as "the art of providing all the context for the task to be plausibly solvable by the LLM" and he is right.

With the rise of Agents it becomes more important what information we load into the "context window memory". We are seeing that the main thing that determines whether an Agent succeeds or fails is the quality of the context you give it. Most agent failures are not model failures anymore, they are context failures.

What is the Context?

To understand context engineering, we must first expand our definition of "context." It isn't just the single prompt you send to an LLM. Think of it as everything the model sees before it generates a response.



IBM TechExchange Training Community Credentials Events Conference

Log In

ISV Ecosystem

Engage with the community of IBM Z and LinuxONE independent software vendors: share knowledge, exchange ideas, explore innovation.

Group Home Threads 100 Blogs 100 Upcoming Events 0 Library 42 Members 1.2k

View Only

Back to Blog List

Why Context Engineering is the New Buzzword in AI and Why It Matters

By Philip O'Shea posted Tue September 04, 2023 04:24 AM

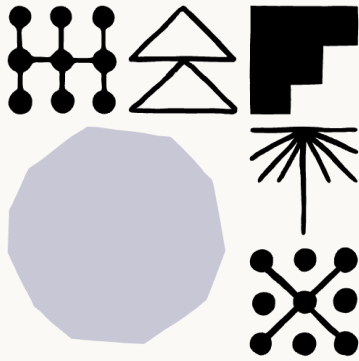
User Interactions External Data

AI

Research Economic Futures Commitments Learn News

Try Claude

Engineering at Anthropic



Effective context engineering for AI agents

Published Sep 29, 2025

Context is a critical but finite resource for AI agents. In this post, we explore strategies for effectively curating and managing the context that powers them.

Prompt Engineering Guide

Courses About Search

Services

Agents

Guides

Context Engineering Guide

Context Engineering Guide

Table of Contents

- What is Context Engineering?
- Context Engineering in Action
 - System Prompt
 - Instructions
 - State Transf
 - Structured Outputs and Outputs
 - Tools
 - RAG & Memory
 - States & Historical Context
 - Advanced Context Engineering
 - Resources

What is Context Engineering?

Google for Developers

Search all articles...

Search

Context engineering

— treating context as a first-class system with its own architecture, lifecycle, and constraints.

Based on our experience scaling complex single- or multi-agentic systems, we designed and evolved the context stack in **Google Agent Development Kit (ADK)** to support that discipline. ADK is an open-source, multi-agent-native framework built to make **active context engineering** achievable in real systems.

The scaling bottleneck

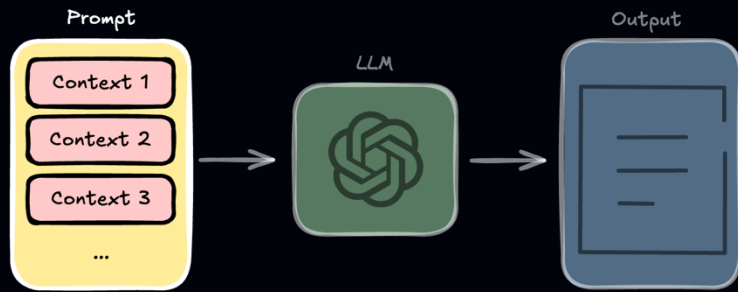
A large context window will help context-related problems but won't address all context-related problems. In practice, the naive pattern—**append everything into one giant prompt**—collapses under a three-way pressure:

- Cost and latency spirals:** Model cost and time-to-first-token grow quickly with context size. "Shoveling" raw history and verbose tool payloads into the window makes agents prohibitively slow and expensive.
- Signal degradation** ("lost in the middle"): A context window flooded with irrelevant logs, stale tool outputs, or deprecated state can distract the model, causing it to fixate on past patterns rather than the immediate instruction. To ensure robust decision-making, we must maximize the density of relevant information.
- Physical limits:** Real-world workloads—involving full RAG results, intermediate artifacts, and long conversation traces—eventually overflow even the largest fixed windows.

Context Engineering is Not Prompt Engineering

When we send a prompt to a Large Language Model (LLM), it produces a response. The quality of that response depends heavily on what we send - AKA the various pieces of context included in the prompt.

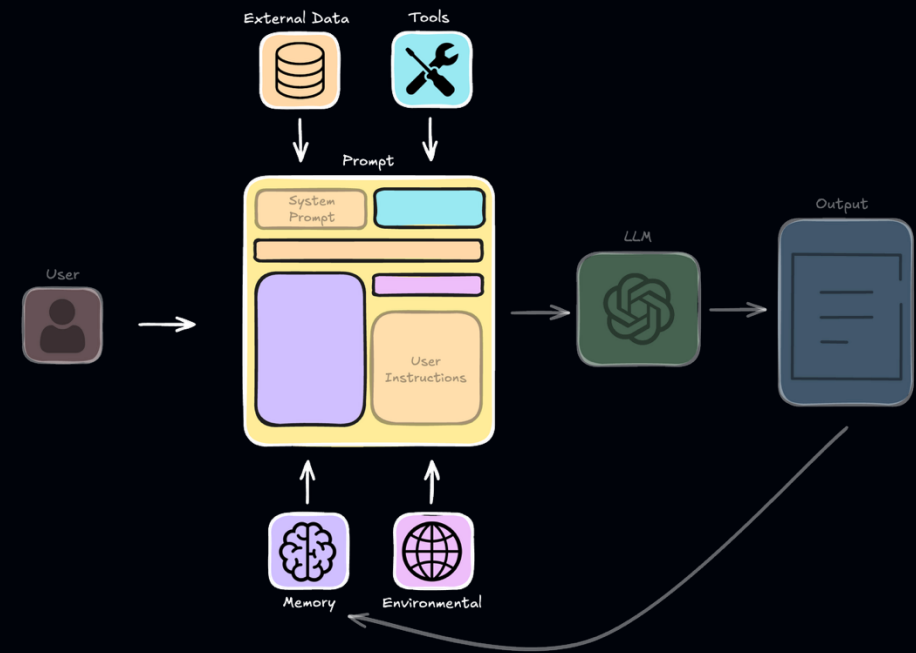
Prompt Engineering



In **prompt engineering**, we craft a static prompt that contains all the information needed.

But agents don't work like that - the same agent needs different pieces of context from different sources depending on the task...

Context Engineering

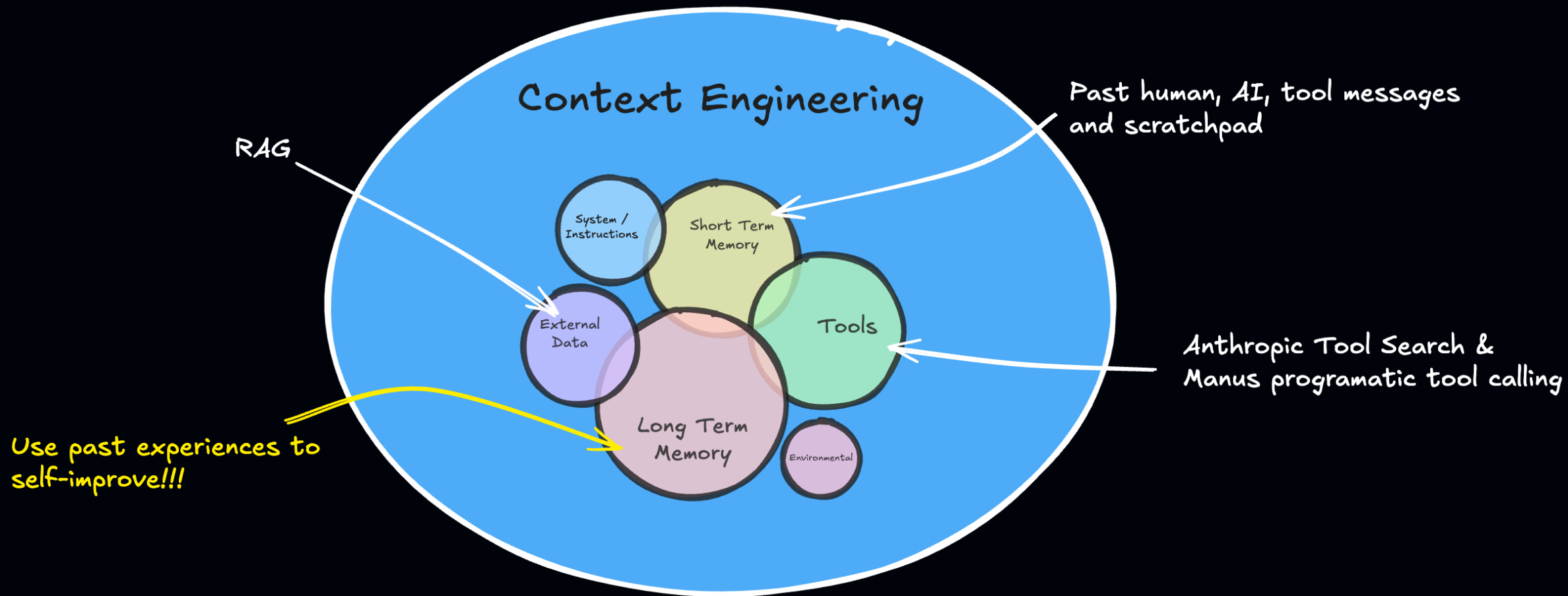


In **context engineering**, we build a system that dynamically manages & injects only the most useful context for the current task into the prompt.

Additionally, this system can store and recall useful memories to guide future interactions.

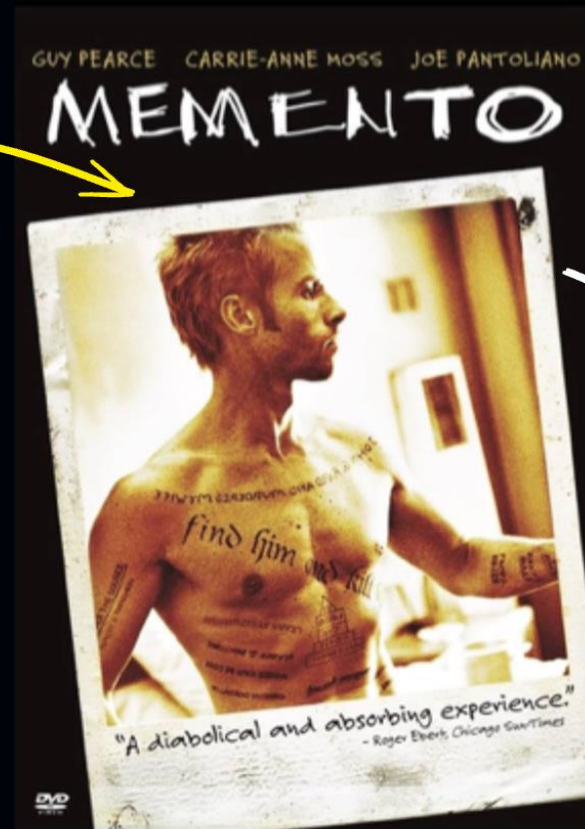
Context engineering is the art of building a system to provide just the right amount of context, from the right sources at the right time.

Types of Context



Without Long-Term Memory

Your AI system is
like the guy from Memento...



Every time you start a new session,
it forgets everything which it "learnt"
in the past session

It cannot reliably self-improve

Long-Term Memory

In the Voyager paper, Minecraft AI agents are allowed to continuously create their own code functions to get new abilities such as using a sword



Semantic Memory

- This is what we know (learnt facts)
- Includes memories such as “the user’s name is Jacob. He likes...”
- Learning in semantic memory involves refining the semantic memory knowledge base: adding new info, updating existing memories, broadening covering new info



Episodic Memory

- This is what we remember experiencing
- Includes memories of what went well / badly such as “user did not like my use of emojis, I should not use them in future”.
- Learning in semantic memory involves refining the semantic memory knowledge base: adding new info, updating existing memories, broadening covering new info



Procedural Memory

- This is the most abstract memory type. It covers how we remember to perform tasks instinctively like riding a bike or typing on a keyboard
- In an agent, this is the LLM weights, the code of the system and the system prompt.
- Learning involves letting the agent fine-tune itself or change its source code such as **Voyager paper**

Benefits of Long-Term Memory

1. Self-Improvement

- Can learn how to handle edge cases they have tackled in the past
- Can learn from humans in the loop so it never needs to make the same request twice

2. Improved Efficiency

- Avoid trying things that have not worked in the past such as specific arguments in tool calls

3. Deep Personalization

- Agents can remember user preferences, history, and style, tailoring interactions to each individual
- Can increase product stickiness

But We Need to Carefully Manage Long-Term Memory & Other Context

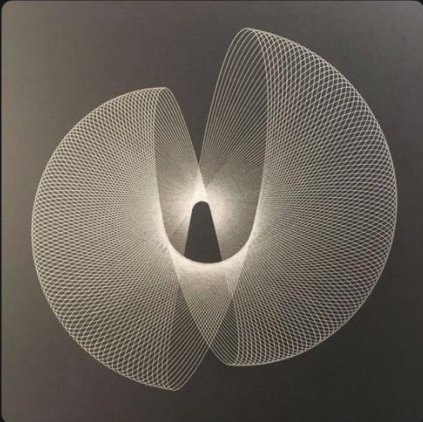
CHROMA TECHNICAL REPORT
July 14, 2025

Context Rot: How Increasing Input Tokens Impacts LLM Performance

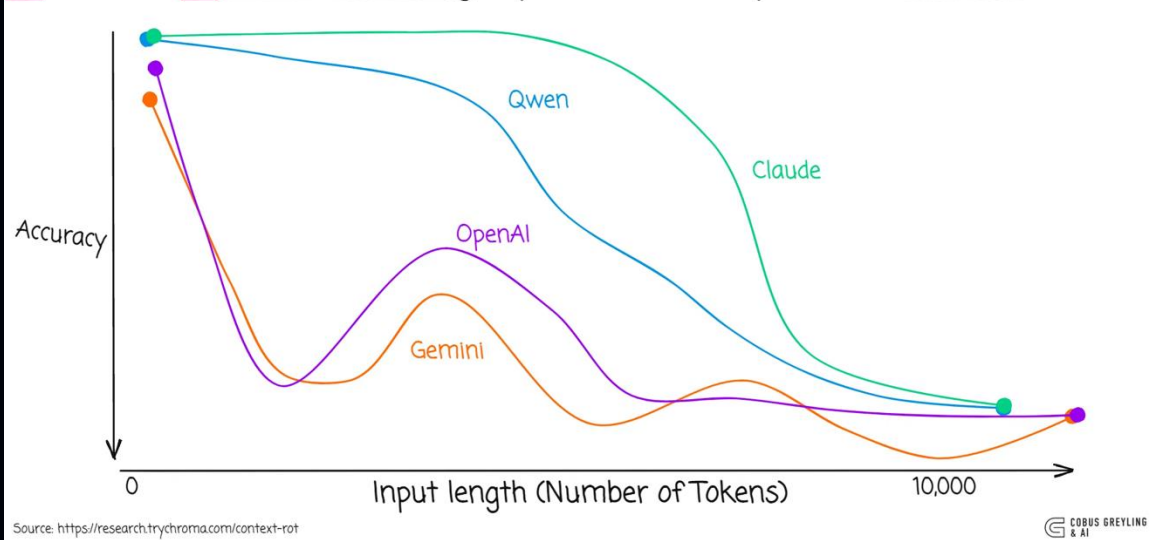
Kelly Hong Researcher - Chroma
Anton Troynikov Cofounder, Advisor - Chroma
Jeff Huber Cofounder, CEO - Chroma

Large Language Models (LLMs) are typically presumed to process context uniformly—that is, the model should handle the 10,000th token just as reliably as the 100th. However, in practice, this assumption does not hold. We observe that model performance varies significantly as input length changes, even on simple tasks.

In this report, we evaluate 18 LLMs, including the state-of-the-art GPT-4.1, Claude 4, Gemini 2.5, and Qwen3 models. Our results reveal that models do not use their context uniformly; instead, their performance grows increasingly unreliable as input length grows.



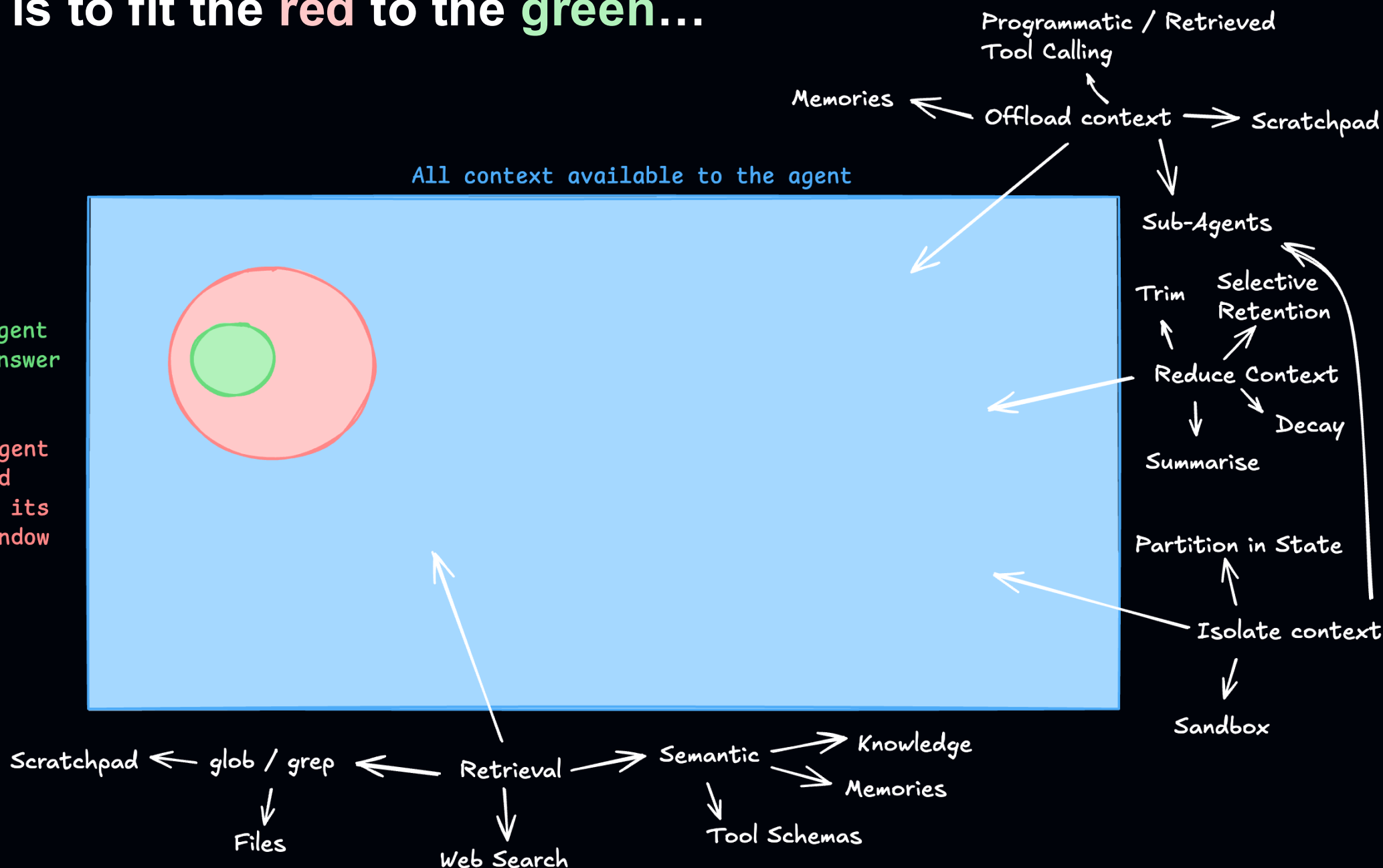
Context Rot: How Increasing Input Tokens Impacts LLM Performance



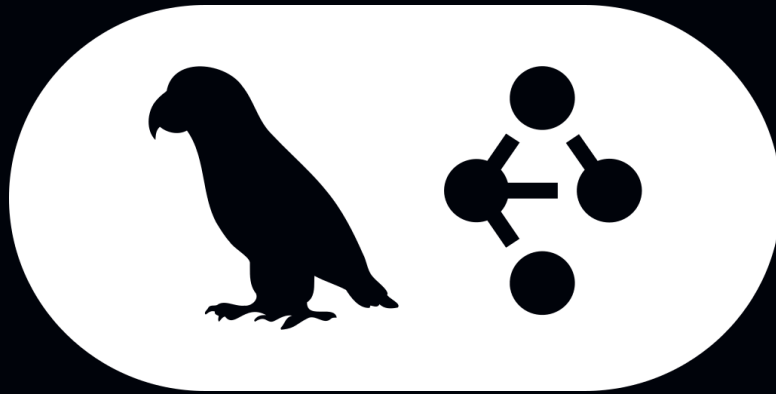
Our job is to fit the red to the green...

What the agent
needs to answer
question X

What the agent
selects and
pulls into its
context window



LangGraph & LangChain handle HITL,
streaming, persistence, memory, error handling,
provider wrapping, type enforcing, control flow...



Let's see how we can do all this
with **LangGraph**

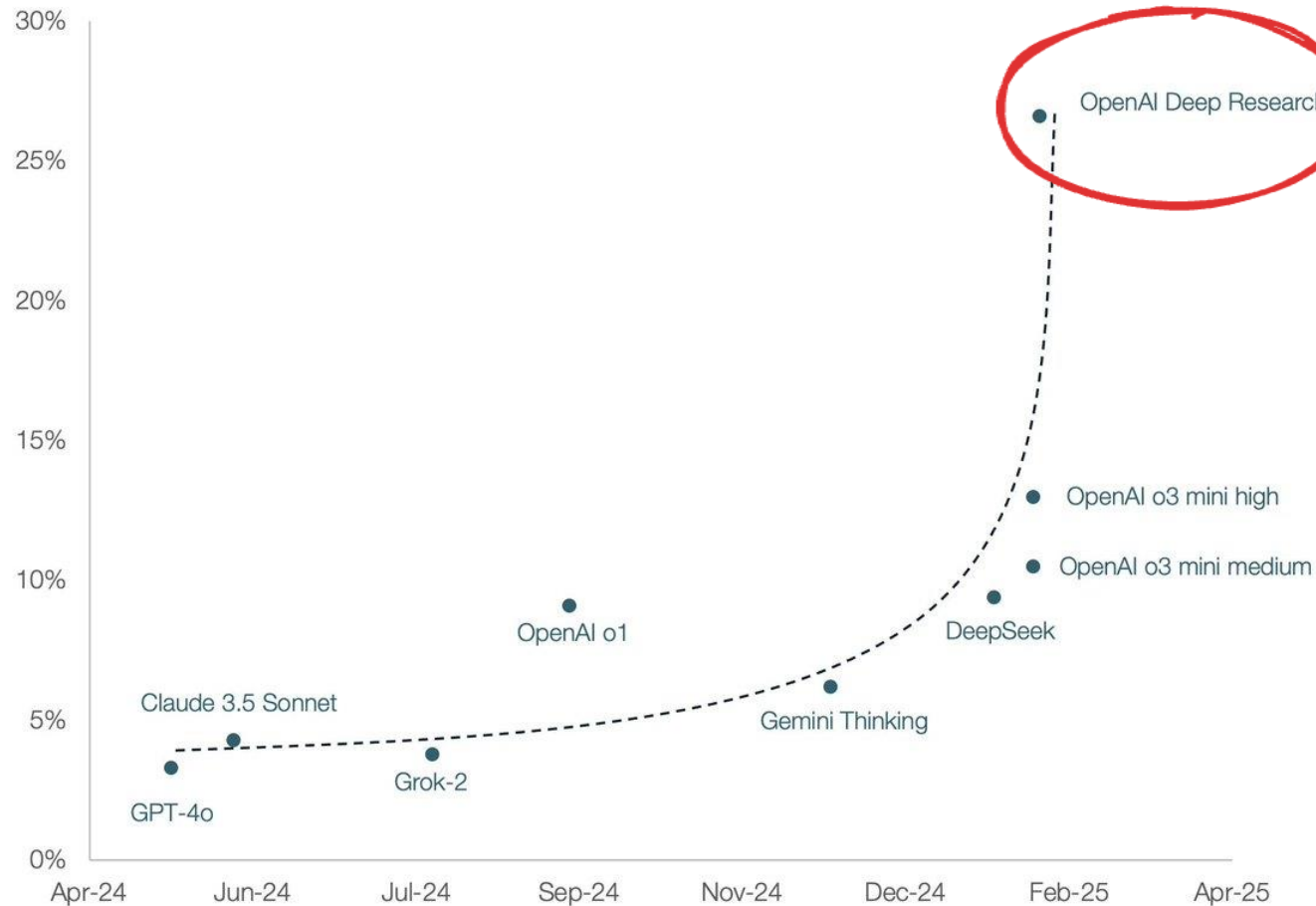
Code at: github.com/JacobShort11/deep-agents-llm-memory-langgraph

Once intelligence reaches a threshold, multi-agent systems become a vital way to scale performance.

For instance, although individual humans have become more intelligent in the last 100,000 years, human societies have become exponentially more capable in the information age because of our collective intelligence and ability to coordinate.

Even generally-intelligent agents face limits when operating as individuals; groups of agents can accomplish far more.

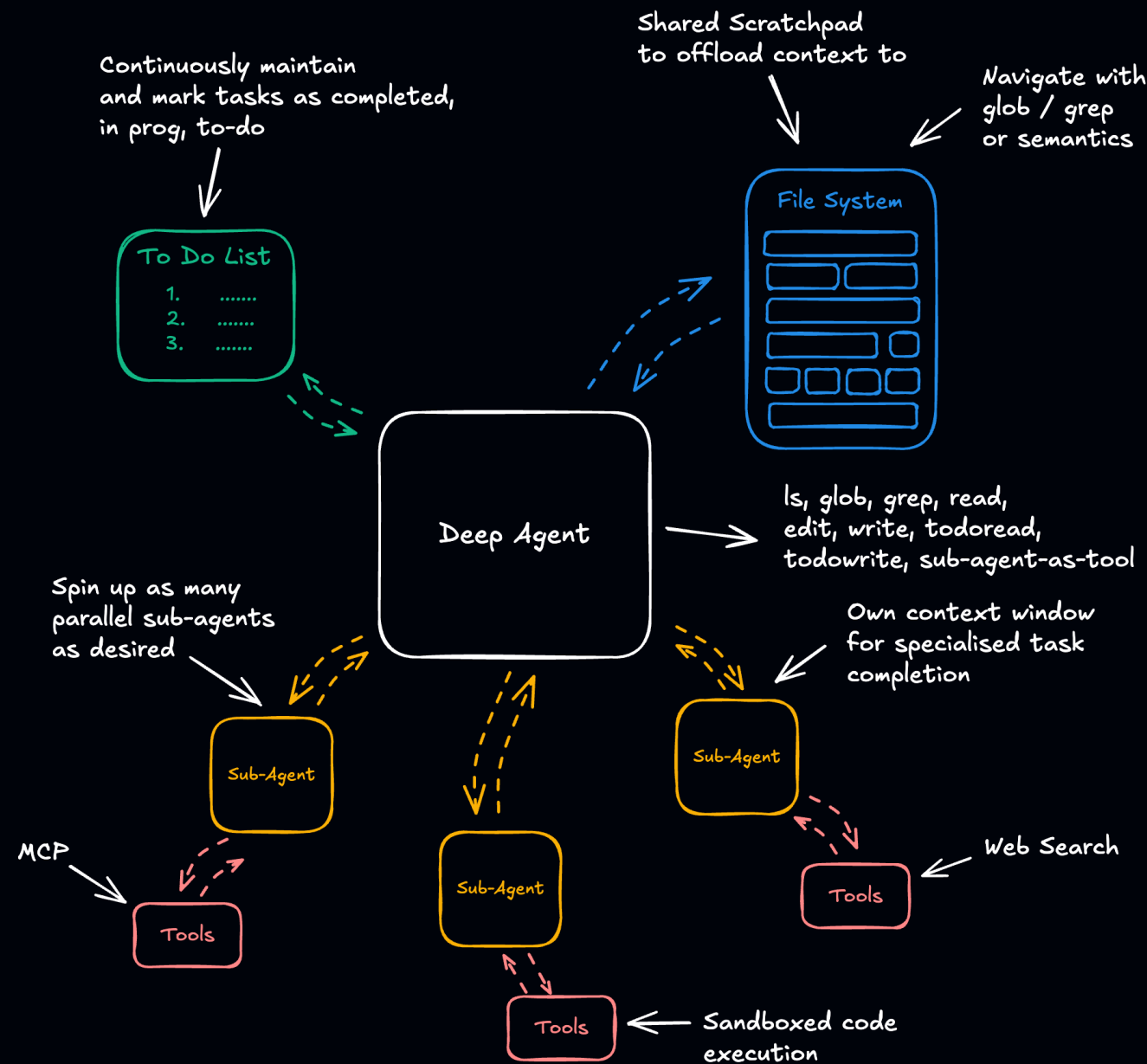
AI Scores on *Humanity's Last Exam*



Source: Tomas Pueyo for Uncharted Territories, with data from Dan Hendrycks, of Humanity's Last Exam

Multi-agent systems with Claude Opus 4 as the lead agent and Claude Sonnet 4 subagents outperformed single-agent Claude Opus 4 by 90.2%

Deep Agents



Benefits of Deep Agents

For multi-step, complex problems requiring adaptive long-horizon planning, multi-hop retrieval, iterative tool use like a human analyst...

Improved Focus

...if a step fails, it doesn't just retry blindly, it updates the plan to accommodate the failure. This keeps the agent focused on the high-level task.

Better Performance

...go deep into tasks by splitting them into sub-tasks and offloading these tasks to sub-agents, each task gets an agent's full attention. The resulting analysis can be saved to the file system instead of overloading the main agent's context window reducing context rot.

Infinite Context

...scratchpad, which can be accessed at any moment, negates the need for larger context windows.

Parallelized Speed

...sub-agents (performing independent tasks) are called in parallel resulting in 80+% speed improvements.

Cross-Referenced & Fact-Checked Information

...credibility sub-agents can cross-reference every claim against multiple sources and provide citations.

Specialized Task Execution

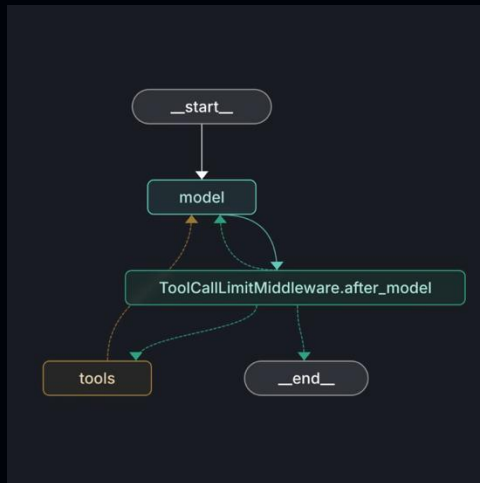
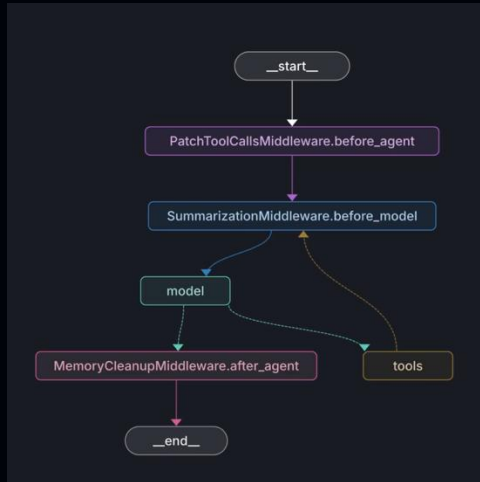
...sub-agents each have different specialties such as code execution for time-series analysis, web research, internal data fact finding, source accuracy checking.

Better Retrieval

...using glob / grep instead of / with semantic search

Reusability

...sub-agents can be reused across different systems (or even retrieved from company-wide repositories)



Let's build a self-improving **deep agent** with long-term memory

Code at: github.com/JacobShort11/deep-agents-lt-memory-langgraph

Demo Outputs

Research sub-agents investigate markets in real-time

Credibility agent cross-references every fact and claim to be true

Executive Summary

Your book is viewed to the AI/robotics cycle vs NADA and US Banks, which remain the dominant positive drivers but carry rising regulatory and China policy headwinds risk. The AI/robotics cycle is high growth and high growth through 2025, but the AI/robotics cycle is a hot topic, with BlackRock very prominently and large institutions concerning the stock. Value Proposition: BlackRock's 2025 investment strategy, which is a hot topic, with BlackRock very prominently and large institutions concerning the stock. Value Proposition: BlackRock's 2025 investment strategy, which is a hot topic, with BlackRock very prominently and large institutions concerning the stock.

Findings

1. Single-Name Equity Exposures

NADA (NADA) - Long \$5.0m

Key 12-month drivers

- AI/robotics super-cycle remains intact, earnings still compounding at extreme rates.
- Q1 FY25 (Aug 2024) revenue \$20.0B, +125% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$20.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).

Implications

- AI/robotics super-cycle remains intact, earnings still compounding at extreme rates.
- Q1 FY25 (Aug 2024) revenue \$20.0B, +125% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$20.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).

Recommendations

- AI/robotics super-cycle remains intact, earnings still compounding at extreme rates.
- Q1 FY25 (Aug 2024) revenue \$20.0B, +125% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$20.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).

Analysis sub-agent generates visual & metric analysis

Very high correlation with growth metrics:

- NADA vs US Banks: $\rho = 0.95$
- AI/robotics super-cycle: $\rho = 0.95$
- Low debt metrics: $\rho = 0.95$
- US Banks vs US Banks: $\rho = 0.95$
- AI/robotics super-cycle vs US Banks: $\rho = 0.95$
- Low debt metrics vs US Banks: $\rho = 0.95$
- AI/robotics super-cycle vs Low debt metrics: $\rho = 0.95$
- Low debt metrics vs AI/robotics super-cycle: $\rho = 0.95$

4. Gradient Boosting Time-Series Model - NVIDIA (Capability Demo)

Interpretation

Analysis sub-agent generates visual & metric analysis

Key 12-month drivers

- AI/robotics super-cycle remains intact, earnings still compounding at extreme rates.
- Q1 FY25 (Aug 2024) revenue \$20.0B, +125% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$20.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).

Implications

- AI/robotics super-cycle remains intact, earnings still compounding at extreme rates.
- Q1 FY25 (Aug 2024) revenue \$20.0B, +125% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$20.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).

Recommendations

- AI/robotics super-cycle remains intact, earnings still compounding at extreme rates.
- Q1 FY25 (Aug 2024) revenue \$20.0B, +125% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$20.0B (+104% YoY).
- Q1 FY25 (Jan 2024) revenue \$28.0B, +104% YoY; data-center revenue \$28.0B (+104% YoY); revenue \$28.0B (+104% YoY).

Analysis sub-agent completes a complete time-series model training and eval run with sandboxed code forecast future price movements

Very high correlation with growth metrics:

- NADA vs US Banks: $\rho = 0.95$
- AI/robotics super-cycle: $\rho = 0.95$
- Low debt metrics: $\rho = 0.95$
- US Banks vs US Banks: $\rho = 0.95$
- AI/robotics super-cycle vs US Banks: $\rho = 0.95$
- Low debt metrics vs US Banks: $\rho = 0.95$
- AI/robotics super-cycle vs Low debt metrics: $\rho = 0.95$
- Low debt metrics vs AI/robotics super-cycle: $\rho = 0.95$

4. Gradient Boosting Time-Series Model - NVIDIA (Capability Demo)

Interpretation

Analysis sub-agent completes a complete time-series model training and eval run with sandboxed code forecast future price movements

Charts and Analysis

1. Macro Backdrop (Yields, USD, Oil, Financials)

Interpretation

- 2Y and 10Y yields are mostly lower than historic, setting a pivot from "higher for longer" to gradual easing.
- USD index has hit a long-term low, consistent with US growth outperformance but expectations of a strong Fed easing.
- Oil prices are slightly down over the period, reflecting oversupply versus despite geopolitical risks.
- US Financials are not too strong from high beta growth, aligned with solid AI and flows but offset by higher expenses and credit concerns.

2. JPM vs NVIDIA vs AAPL

Interpretation

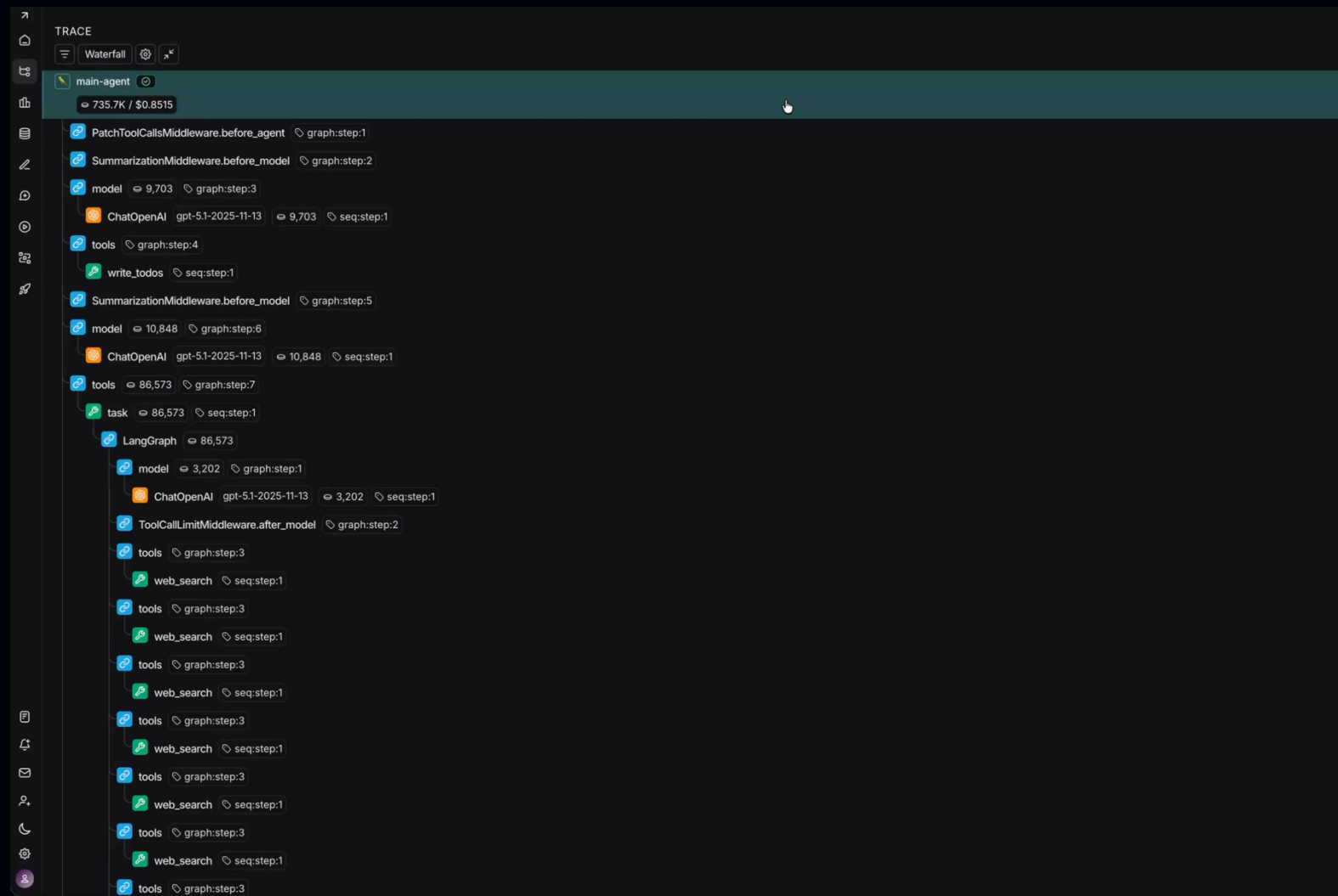
- 2Y and 10Y yields are mostly lower than historic, setting a pivot from "higher for longer" to gradual easing.
- USD index has hit a long-term low, consistent with US growth outperformance but expectations of a strong Fed easing.
- Oil prices are slightly down over the period, reflecting oversupply versus despite geopolitical risks.
- US Financials are not too strong from high beta growth, aligned with solid AI and flows but offset by higher expenses and credit concerns.

3. Cross-Asset Correlation Heatmap

Analysis sub-agent completes a complete time-series model training and eval run with sandboxed code forecast future price movements

Time Elapsed: 6 minutes (constrained tool limits for cost reasons - previously have had this agent run for 1+ hours)
Sub-Agents Spawned: 21
Web Queries: 315
Code Scripts Executed: 7

Demo Trace



Small Part of Deep Agent Trace Showing How Many Actions it Performs In Minutes

Demo Inner Workings

Maintain to-do list

Spawn sub-agents in parallel

Navigate files & memories

Edit files & memories

Credibility agent cross-references every fact and claim to be true

Sandboxed code execution

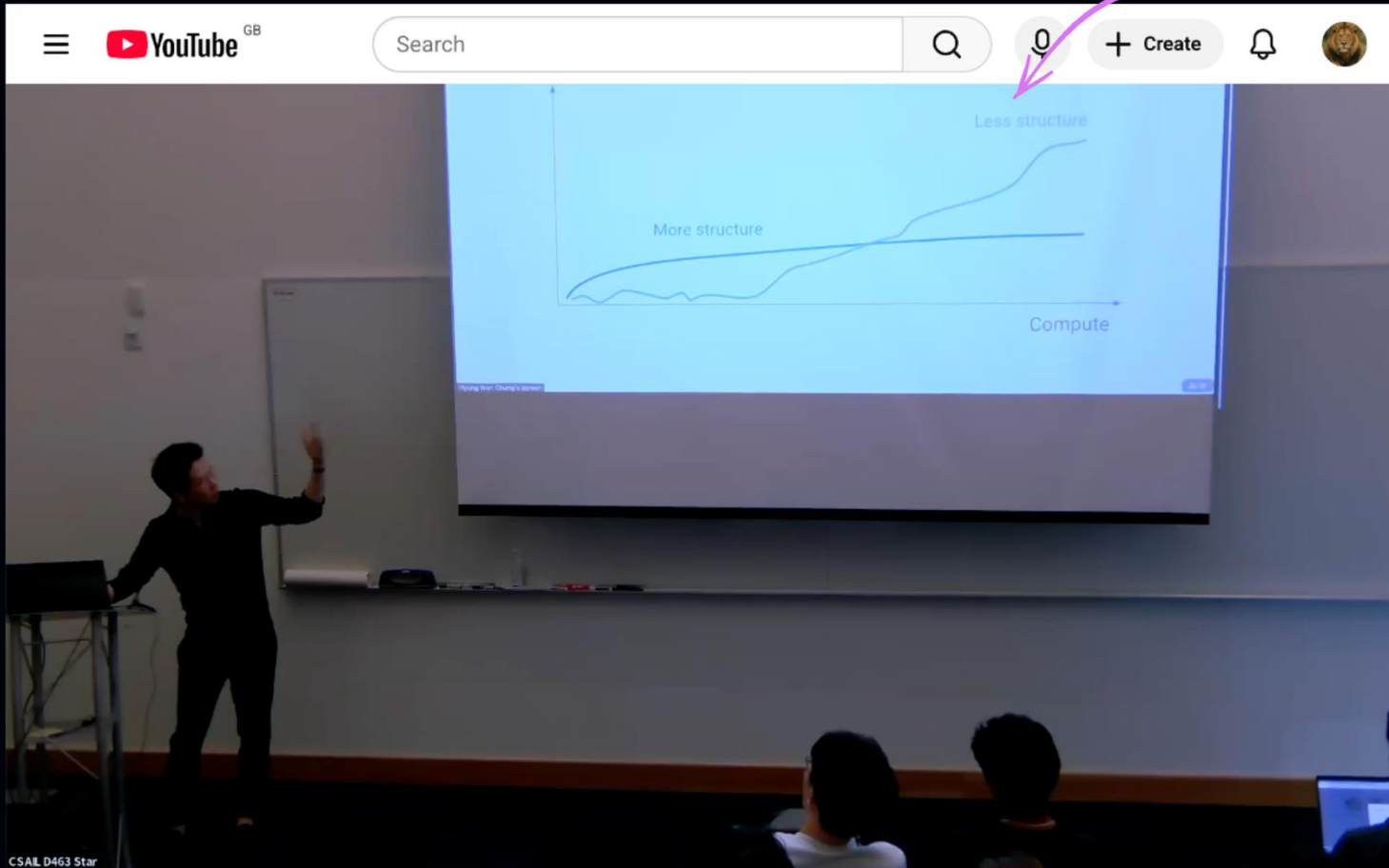
Web search

Less Structure = More Performance

Harder governance ↙
Evals more important ↘

As LLMs get smarter, removing the structure we imposed to make them behave previously allows for performance gains

This means less linear workflows, more autonomy (i.e. deep agents)



Manus test their system with SLMs... if performance not greater with LLMs then too much structure has been imposed

MIT EI seminar, Hyung Won Chung from OpenAI. "Don't teach. Incentivize."

Long-Term Memory Use Cases

This is single easiest, most effective use of long-term memory right now



User Preference / Knowledge

- Increase stickiness by adapting outputs to individual user preferences, terminology, and working style.
- Remember important information about user to reuse in future responses.



Self-Improvement

- Learns directly from end-user feedback, prioritising approaches that have historically been successful.
- Discovers & adopts best arguments to use in tool calls.



Multi-Session Memory

- Preserves context across long-running user tasks, remembering decisions, assumptions, and prior outputs.
- Makes AI feel like a persistent collaborator, increasing trust and repeat usage.

Deep Agent Use Cases



Deep Research over Sensitive Data

- Many companies IP is their data, this cannot be uploaded to copilot deep research etc...
- They need custom MCP retrieval tools building for them and custom deep agents with specialized capabilities for optimal performance.
- There deep agents could be running in background instead of triggered via chat interfaces.



Legacy Code Conversion

- Deep agent coding agents (i.e. Claude code) are SOTA.
- These do not handle legacy / unusual code well in line with company guidelines (and have data privacy problems).
- In-house tailored solutions needed.

1. Extract top 300 news sources of last 3 hours and turn into single document capturing sentiment towards i.e. a market

2. BERT classifier to map document to numeric metrics (representing overall sentiment)

3. Use these metrics in downstream quant models to improve price / volatility / risk predictions

...

Experimental (Something New)

Create research slide deck using python-pptx

Create podcast to explain research using NotebookLM podcast API

Takeaways

1. Long-Term Memory is a Simple & Ultra-Effective Way to Improve AI Solutions

2. Deep Agents Allow us to Solve More Complex Long-Time-Horizon Tasks

3. LangGraph Is Amazing & Used By:    ...



github.com/JacobShort11/deep-agents-lt-memory-langgraph

- Deep agent code [~4000 lines of code]
- LangGraph code demos [~500 lines of code]
- LangChain x LangGraph capability notes and demos [~100 pages]